ChatGPT

에이전틱 워크플로우 오케스트레이션: Loopy Era부터 Agentic Engineering까지

변화의 핵심 요약

2025년 말~2026년 초에 걸쳐 "코딩을 돕는 AI"에서 "코딩(그리고 더 넓은 작업)을 수행하는 에이전트"로 중심이 이동하면서, 개발자의 병목이 **타이핑 속도나 compute**가 아니라 **오케스트레이션(업무 분해·경계조건·평가·검증·운영)**으로 바뀌고 있다는 관측이 빠르게 확산됐다. 이를 Karpathy는 여러 인터뷰/프로젝트에서 "기본 워크플로우가 (2025년) 12월을 기점으로 뒤집혔다"는 식으로 설명했고, 실제로 "12월 이후 직접 코드를 한 줄도 안 쳤다"는 말까지 했다. [1]

이 변화를 이해하는 데 유용한 키워드가 질문에 정리된 다섯 가지다:
(1) Loopy Era(루프 자체가 제품/연구의 단위가 되는 시대), (2) AI Psychosis/Claw Psychosis(가능성에 대한 강박적 초조함), (3) Remove Yourself as the Bottleneck(사람이 루프의 '필수 단계'가 되지 않게 설계), (4) March of Nines(장기·다단계 흐름에서 신뢰성의 기하급수적 붕괴), (5) Agentic Engineering('바이브 코딩' 이후의, 규율 있는 에이전트 기반 개발). [2]

Loopy Era의 실체: "루프를 설계하고 경계조건을 세팅하는 일"이 본업이 되는 이유

AutoResearch가 보여준 '극단적으로 작은 연구 루프' 패턴

Karpathy가 2026년 3월 공개한 GitHub 프로젝트 **autoresearch**는 "가설→코드 수정→짧은 학습/실험→평가→유지/폐기→반복"을 **사람 없이** 굴리기 위해, 리포지토리를 의도적으로 세 개의 핵심 파일로 압축했다. [3]

- `prepare.py`: 데이터 준비·토크나이저·평가 등 **고정**(수정 금지) [3]
- `train.py`: 에이전트가 수정하는 **유일한 실험장**(아키텍처/옵티마이저/하이퍼파라미터/루프 등 "Everything is fair game") [4]
- `program.md`: 인간이 작성/수정하는 **운영 규약(조직의 OS)**—에이전트가 무엇을 할 수 있고/없고, 어떤 지표로 '더 낫다'를 판단하며, 어떤 루프로 계속 돌지까지를 자연어로 명시 [5]

이 구조에서 중요한 점은 "AI가 잘하도록 만들기"보다 "AI가 **반드시** 잘하도록 만들 수 있는 문제 형태로 **경계조건을 좁히는 것**"이다. 예를 들어, `program.md`는 "수정 가능한 파일은 `train.py` 하나뿐"이고, 고정된 5분 시간예산 안에서 `val_bpb`를 낮추는 것이 목표라고 못 박는다. 즉, 사람이 '연구 방향과 제약'을 쓰고, 에이전트는 '실행과 탐색'을 담당한다. [4]

"program.md는 조직을 프로그래밍하는 파일"이라는 관점

Karpathy는 No Priors: Artificial Intelligence [6] 에서 `program.md`를 "auto researcher가 어떻게 일해야 하는지에 대한 (자신의) 미흡한 시도"라고 부르면서, **서로 다른 program.md가 서로 다른 연구 진척을 만든다**는 식으로까지 확장한다. 이는 "사람의 역할 = 루프 설계/경계조건 세팅"이라는 질문의 요약과 정확히 겹친다. [7]

또한 `program.md`에는 "루프가 시작되면 멈추지 말라(NEVER STOP)"처럼, 사람의 확인이 없어도 지속 실행하도록 강하게 지시하는 대목이 포함돼 있다. 이 지점이 'Loopy Era'의 촉감이다: **사람이 루프의 필수 단계가 아닌 것**이 곧 생산성의 본질이 된다. [8]

AI Psychosis / Claw Psychosis: 기술적 한계보다 '오케스트레이션 스킬'이 불안의 원인이 되는 순간

"앞으로 무엇이 가능한지"를 따라잡지 못한다는 초조함

Karpathy는 2026년 3월 공개된 No Priors 인터뷰에서 스스로를 "가능한 것을 알아내기 위해 한계까지 밀어붙이는 'psychosis' 상태"라고 표현하며, 특히 "트위터에서 다른 사람들이 하는 걸 보면 불안하고, 최전선에 있지 않으면 초조하다"는 심리(질문에서 말한 '루프 밖에 있으면 안절부절')를 구체적으로 언급한다. [9]

이 발화가 흥미로운 이유는, 불안의 원인이 "모델이 부족해서"라기보다 "내가 이 새로운 도구들을 **어떻게 병렬로, 어떻게 적절히 돌려야 하는지** 아직 감을 못 잡아서"로 이동한다는 점이다. 실제로 그는 "단일 세션이 아니라 여러 세션(여러 에이전트)을 어떻게 띄우고 운영할지"에 집착하고 있음을 말한다. [7]

Claw Psychosis: '지속 실행(persistence)' 에이전트가 열어젖힌 새로운 강박

동일 인터뷰/보도에서 Karpathy는 2026년 1월 "claw psychosis"를 겪었다고 말하며, 집안의 여러 시스템을 제어하는 "Dobby"라는 claw(지속 실행형 에이전트)를 만들었다고 소개한다. 이 claw는 조명/블라인드/보안/수영장·스파 등 여러 기능을 통합해 제어한다는 서술이 반복적으로 등장한다. [10]

여기서 'claw'는 단순 챗봇이 아니라, **현실 세계의 도구(앱/디바이스/웹/로컬 네트워크)에 연결되어 계속 돌아가는 에이전트**라는 의미로 사용된다. 2026년 2월에는 OpenClaw 창립자인 Peter Steinberger [11] 가 OpenAI [12] 에 합류하고, OpenClaw 프로젝트는 재단 형태로 독립·오픈소스성을 유지하되 OpenAI가 지원한다는 발표도 나왔다. [13]

Remove Yourself as the Bottleneck: "코드는 ephemeral"이고, 인간은 '의지/평가'를 표현하는 역할로 이동

"12월 이후 코드를 안 쳤다"가 의미하는 것

Karpathy의 "12월 이후 코드를 한 줄도 안 쳤다"는 발언은 단순한 과장이라기보다, **업무 분업 구조가 바뀌었다**는 신호다. 그는 "이전에는 (자신이) 대부분을 직접 작성하고 일부만 에이전트에 맡겼는데, 2025년 12월에 비율이 뒤집혔고, 그 뒤로는 더 극단으로 갔다"고 설명한다. [1]

이때 핵심 병목은 "컴퓨트"가 아니라 **사람이 의도를 얼마나 정확히 전달하고, 경계를 어떻게 설정하며, 산출물을 얼마나 빨리 검증/교정하느냐**로 이동한다. Simon Willison은 이를 "코딩 에이전트(코드를 쓰고 실행할 수 있는 에이전트)와 함께 소프트웨어를 개발하는 실천"으로 정의하며, 인간이 해야 할 일이 오히려 '매우 많다'고 강조한다. [14]

'에이전트 하네스(harness)'가 곧 제품: 실행·검증 루프가 내장된 도구들

이 변화는 최근 CLI/IDE 도구 문서에서도 정면으로 드러난다.

- Claude Code는 작업이 "컨텍스트 수집 → 액션 → 검증"의 루프를 반복하며 진행된다고 설명하고, 파일 조작/실행/웹 검색 등 도구 사용이 다시 루프에 피드백으로 들어간다고 명시한다. 또한 프로젝트 지침을 담는 `CLAUDE.md` 같은 파일 기반 컨텍스트를 공식 기능으로 제공한다. [15]
- OpenAI의 Codex CLI 문서는 "선택한 디렉터리에서 코드를 읽고/수정하고/실행"할 수 있는 터미널용 코딩 에이전트라고 정의하며, 서브에이전트·승인(approval) 모드·MCP 연동 등 '오케스트레이션 요소'를 핵심 기능으로 나열한다. [16]
- Google의 Gemini CLI 문서는 CLI가 ReAct(reason-and-act) 루프로 복잡한 작업(버그 수정, 기능 추가, 테스트 커버리지 개선 등)을 수행하며, 로컬/원격 MCP 서버를 통한 도구 확장도 전제로 한다. [17]

즉, “코드는 일회용(ephemeral)”이라는 표현은 (완전히 새 코드를 계속 생성한다는 의미라기보다) **사람이 직접 타이핑하던 구현 부담이 에이전트 루프 안으로 흡수되고**, 사람의 가치는 ‘설계·검증·운영 규약’ 쪽으로 이동한다는 뜻으로 읽힌다. 이 방향성은 “바이브 코딩=무검토 코드 생성”과 “에이전틱 엔지니어링=규율 있는 감독/검증”을 구분하려는 논의에서도 반복된다. [18]

March of Nines: 장기 에이전틱 워크플로우의 신뢰성(AgentOps)의 본질적 난제

왜 “90% 데모”가 프로덕션에서 붕괴하는가

에이전틱 워크플로우는 보통 “의도 파싱→검색/RAG→플래닝→툴 호출→검증→출력 포맷팅…”처럼 **다단계**다. 이때 각 단계가 독립적으로 90% 성공해도, 전체 성공률은 대략 $0.9^{10} \approx 34.9\%$로 급락한다는 ‘복리 실패(compounding failure)’가 핵심 문제로 지적된다. VentureBeat는 이를 Karpathy의 “March of Nines” 프레이밍과 함께, 10단계에서 90%는 34.87%, 99%도 90.44% 수준이라는 표로 설명한다. [19]

Karpathy가 다른 인터뷰에서 “90%는 첫 번째 nine일 뿐이고, 다음 nine들을 쌓는 게 같은 양의 일”이라는 취지로 말한 것으로 인용되기도 한다. [20]

‘nines’를 올리는 실전 레버: 제약(contracts)·그래프·평가·관측

신뢰성을 올리는 방법은 요약하면 “확률적 모델 호출을 **분산시스템처럼 다루는 것**”이다. 핵심 레버들은 업계 글에서 비교적 일치한다:

- 워크플로우를 그래프/상태기계로 고정해 “허용된 상태/툴/재시도”를 명시한다(예: LangGraph는 상태(State)·노드·엣지로 에이전트 흐름을 그래프로 모델링한다고 설명). [21]
- 입출력 계약(스키마 검증), 툴 타임아웃/재시도, 위험도 기반 라우팅(중요 작업은 더 강한 모델·사람 승인) 같은 운영 장치를 덧댄다. [22]
- 무엇보다 “평가(evals)와 관측(observability)”이 있어야 한다. OpenAI는 evals를 “여러 모델·프롬프트 변경 시, 기대 기준을 만족하는지 검증하는 필수 구성요소”로 문서화한다. [23]
- 오픈소스 진영에서는 promptfoo(로컬/CI 기반 평가·레드팀)나 Langfuse(오픈소스 관측/평가), W&B Weave(관측/평가 플랫폼) 같은 도구가 ‘nines 쌓기’의 기반으로 자주 사용된다. [24]

지금 “오케스트레이션을 잘하는 사람/팀”이 쓰는 툴 지형도와 학습용 GitHub 레퍼런스

에이전트 하네스의 3대 UI: 터미널·IDE·클라우드 워커

현재 주류는 “에이전트 하네스 + 룰/스킬 파일 + 승인/샌드박스” 조합으로 수렴하는 경향이 있다.

터미널 기반(가장 ‘루피’하게 굴리기 쉬움): - Claude Code: 코드베이스 전체 맥락, 파일 편집/명령 실행/테스트, 스킬·훅·서브에이전트·MCP 확장 등 “하네스 기능”을 공식 문서로 체계화했다. [25]
- Codex CLI: 로컬에서 읽기/수정/실행 + 서브에이전트 + 승인 모드 + MCP 확장을 표준 기능으로 내세운다. [26]
- Gemini CLI: ReAct 루프 + MCP 서버 연동을 전제로 하는 오픈소스 터미널 에이전트다. [27]

IDE 기반(팀 컨텍스트/규칙을 강제하기 쉬움): - Cursor의 “Agent”는 여러 파일 수정·터미널 실행 등 독립적 작업 수행을 목표로 하며, “Rules”를 시스템 레벨 지침으로 제공해 팀 워크플로우를 공유/관리한다고 설명한다. 이런 룰 파일 생태계(awesome-…-rules)도 이미 커졌다. [28]

클라우드 워커/에이전트 플랫폼(병렬화·지속 실행에 유리): - OpenHands는 SDK/CLI/로컬 GUI/클라우드 배포까지 포괄하며 "로컬에서부터 수천 개 에이전트까지 확장"을 표방한다. 연구 논문(OpenReview)에서도 코드 작성/CLI/웹 브라우징과 더불어 "샌드박스 환경에서의 안전한 실행"과 "평가 벤치마크 통합"을 핵심으로 든다. [29]

자동화된 "리포지토리 단위" 작업을 배우기 좋은 오픈소스 레퍼런스

학습/실험에 특히 좋은 레포들을, 질문의 다섯 개 키워드(루프·경계조건·평가·병렬화·신뢰성) 관점에서 묶어보면 다음과 같다.

Karpathy의 Loopy Era 원형(연구 루프, program.md 패턴): - `karpathy/autoresearch`: "세 파일 랩(prepare/train/program)" + 고정 시간예산 + keep/discard 루프를 통해, '경계조건이 곧 성능'이라는 것을 체감하기 좋다. [4]

소프트웨어 이슈 해결(장기 코딩 에이전트, 평가 기반 최적화): - `SWE-agent/mini-swe-agent`: "bash만으로도 되는" 극단적으로 단순한 에이전트 스캐폴딩과, 샌드박스 실행/디버깅 용이성에 대한 설계를 강조한다. SWE-bench Verified에서 74%+ 성능을 주장하며, 다양한 격리 실행환경(도커 등)을 지원한다고 밝힌다. [30]
- SWE-bench Verified 자체: 실제 GitHub 이슈/PR 기반 평가 셋이며, 오픈AI가 "더 신뢰성 높은 SWE-bench의 인간 검증 서브셋"으로 소개한 바 있다. [31]

멀티 에이전트 오케스트레이션(그래프/이벤트 드리븐): - LangGraph: "장기 실행·상태 유지형 에이전트"를 위한 저수준 오케스트레이션 프레임워크를 표방하고, 상태/노드/엣지 기반 그래프 모델을 문서화한다. [32]
- Microsoft AutoGen / Agent Framework: 멀티 에이전트 앱/워크플로우 구축을 위한 프레임워크로, 이벤트 드리븐 멀티 에이전트 시스템 같은 개념을 전면에 둔다(단, AutoGen은 Agent Framework로의 이동을 권고하는 공지가 함께 있다). [33]
- crewAI: 역할 기반 멀티 에이전트 오케스트레이션을 내세우는 프레임워크로, 공식 블로그/레포에서 "첫 crew 만들기" 등 온보딩 자료를 제공한다. [34]
- `openai/swarm`: "가벼운 멀티 에이전트 오케스트레이션"을 탐구하는 교육용 프레임워크로 소개된다(최근 OpenAI Agents SDK가 Swarm의 'production-ready upgrade'라고 문서화). [35]

룰/체크리스트를 "리포 안에" 넣어 신뢰성을 끌어올리는 접근: - Continue: PR마다 AI 체크를 돌리며, 각 체크가 리포 안의 마크다운 파일로 정의된다고 설명한다(즉, 평가와 정책이 **소스 컨트롤**된다). [36]
- promptfoo: 로컬/CI에서 프롬프트·모델·에이전트의 회귀를 잡기 위한 자동 평가/레드팀 도구를 표방한다. [37]

"Claw" 계열: 지속 실행형 개인 에이전트 생태계(OpenClaw)

질문에 나온 "Claw"라는 단어는 2026년 초 급부상한 오픈소스 개인 에이전트 프로젝트 **OpenClaw**의 밈/생태계와도 연결된다. OpenClaw 레포는 "자신의 디바이스에서 돌리는 개인 AI 어시스턴트"이며, WhatsApp/Telegram/Slack/Discord 등 다양한 채널에서 응답한다고 설명한다. [38]

동시에 "잘못 설정하면 보안 위험/데이터 유출" 우려가 크다는 경고도 빠르게 따라붙었다(Reuters가 중국 당국의 보안 리스크 경고를 전함). [39]
또한 최근에는 OpenClaw/Claude Code 같은 인기 도구를 사칭한 악성코드 유포(검색 광고를 통한 인포스틸러)가 보고되어, "다운로드/설치 경로의 검증"이 실무적으로 중요해졌다. [40]

실전용 오케스트레이션 플레이북: program.md 사고방식으로 "병목 제거 + nines 쌓기"

아래는 질문에 정리된 관점을, 실제 팀/개인이 바로 적용 가능한 형태로 재구성한 "agentic engineering" 체크리스트다(특히 `program.md`/규칙 파일/스킬 파일로 구현 가능).

경계조건을 먼저 '코드화'하라: 허용 범위·금지 범위·정지 조건

autoresearch의 `program.md`는 (1) 수정 가능한 파일, (2) 수정 금지 파일(평가의 심판), (3) 고정된 시간예산, (4) 기록 방식, (5) 루프 지속 조건까지 명시한다. 이런 형태의 "경계조건 문서"가 있어야 Remove Yourself as the Bottleneck이 현실이 된다. [41]

예시 템플릿(개념용):

```
# program.md (template)

## Goal
- 무엇이 "성공"인지 1문장으로 정의

## Scope
- 수정 가능한 파일/디렉터리:
- 읽기 전용(절대 수정 금지):
- 네트워크/외부 API 사용 허용 여부:

## Evaluation
- 자동 채점(테스트/리그레션/메트릭):
- 실패 시 처리(재시도/롤백/사람 호출):
- 결과 기록 포맷(로그/TSV/JSON):

## Safety
- 비밀정보/키/PII 금지 규칙:
- destructive action 금지(삭제/배포/결제 등):
- 승인 모드(항목별 승인 vs 세션 승인):

## Loop
- 1회 iteration의 단계:
- stop 조건(시간/개수/품질/에러 버짓):
```

이 문서의 목적은 "프롬프트를 잘 쓰기"가 아니라, **에이전트가 무엇을 반복해도 '안전하고', 무엇을 반복하면 '개선'인지가 흔들리지 않게** 하는 것이다. Claude Code의 `CLAUDE.md`, Cursor의 Rules, Codex의 "skills" 같은 기능들이 실무에서 같은 문제(지속 컨텍스트·규칙·재사용)를 풀기 위해 등장했다는 점이 중요하다. [42]

테스트/평가가 없으면 'nines'는 오지 않는다

Addy Osmani나 Simon Willison 계열의 "agentic engineering vs vibe coding" 논의에서 반복되는 결론은, 에이전트가 코드 작성 속도를 올려도 **검증(테스트/리뷰/평가) 체계가 없으면 품질은 망가진다**는 것이다. [43]

따라서 March of Nines를 의식한다면, 다음이 우선순위가 된다: - 단계 수 자체를 줄이거나(불필요한 tool call 제거), 단계 간 인터페이스를 스키마로 단단히 묶고(계약 위반 즉시 실패), 실패를 복구 가능한 방식으로 설계한다. [44]
- 평가를 소스 컨트롤로 끌어들인다(예: PR 체크를 마크다운 파일로 정의하는 Continue). [45]
- 관측/트레이싱을 붙여 "왜 실패했는지"를 재현 가능하게 만든다(예: Langfuse는 오픈소스 LLM 엔지니어링 플랫폼으로 모니터링·평가·디버깅을 표방, Weave도 관측/평가 플랫폼을 표방). [46]

병렬 오케스트레이션: “매크로 액션” 단위로 일을 나눠라

Karpathy는 여러 에이전트를 동시에 띄워 “리포지토리에 대해 더 큰 매크로 액션 단위로 조작한다”는 감각을 강조한다. 또한 화면에 다수의 Codex 에이전트를 타일링해 왔다갔다 하는 사람(예: Peter Steinberg)을 언급하며, 병렬화가 곧 숙련의 방향임을 시사한다. [47]

이 아이디어는 tmux/멀티패널 같은 UI 취향을 떠나, 구조적으로는 다음을 의미한다: - 작업을 “서로 간섭하지 않는 기능 덩어리”로 쪼개고
- 각 덩어리를 에이전트에게 위임하되
- 합치기 전에 테스트/요약/리뷰(가능하면 별도 에이전트)를 넣어 “결합 단계”의 실패를 줄인다. [48]

보안·권한·샌드박스는 ‘선택’이 아니라 기본값

에이전틱 도구는 파일 수정·명령 실행·웹 접근을 한다. 따라서 권한/승인 모드가 핵심 UX가 된다. Claude Code는 파일 수정 전 승인(또는 세션 단위 “Accept all”)을 문서화하고, Codex CLI도 승인 모드(approval modes)를 주요 기능으로 둔다. [49]

또한 공격자들이 인기 도구를 사칭해 “터미널에 명령을 붙여넣게” 만드는 방식으로 인포스틸러를 퍼뜨린 사례까지 보고된 만큼, 조직/개인 차원에서 설치 경로·격리 환경·최소권한은 필수가 됐다. [40]

navlist최근 에이전틱 워크플로우 관련 주요 보도turn6news37,turn6news41,turn19news37,turn20news33,turn20news36,turn20news34

---

[1] [2] [7] [9] [11] [47] [48] No Priors: Artificial Intelligence | Technology | Startups - Andrej Karpathy on Code Agents, AutoResearch, and the Loopy Era of AI Transcript and Discussion
https://podscripts.co/podcasts/no-priors-artificial-intelligence-technology-startups/andrej-karpathy-on-code-agents-autoresearch-and-the-loopy-era-of-ai

[3] [4] [5] GitHub - karpathy/autoresearch: AI agents running research on single-GPU nanochat training automatically · GitHub
https://github.com/karpathy/autoresearch

[6] [32] langchain-ai/langgraph: Build resilient language agents as ...
https://github.com/langchain-ai/langgraph?utm_source=chatgpt.com

[8] [41] autoresearch/program.md at master · karpathy/autoresearch · GitHub
https://github.com/karpathy/autoresearch/blob/master/program.md

[10] OpenAI cofounder says he hasn't written a line of code in months and is in a 'state of psychosis' | Fortune
https://fortune.com/2026/03/21/andrej-karpathy-openai-cofounder-ai-agents-coding-state-of-psychosis-openclaw/

[12] [15] [42] How Claude Code works - Claude Code Docs
https://code.claude.com/docs/en/how-claude-code-works

[13] [39] OpenClaw founder Steinberger joins OpenAI, open-source bot becomes foundation | Reuters
https://www.reuters.com/business/openclaw-founder-steinberger-joins-openai-open-source-bot-becomes-foundation-2026-02-15/

[14] What is agentic engineering? - Agentic Engineering Patterns - Simon Willison's Weblog
https://simonwillison.net/guides/agentic-engineering-patterns/what-is-agentic-engineering/

16 26 CLI – Codex | OpenAI Developers
https://developers.openai.com/codex/cli/

17 27 Gemini CLI | Gemini Code Assist | Google for Developers
https://developers.google.com/gemini-code-assist/docs/gemini-cli

18 43 AddyOsmani.com - Agentic Engineering
https://addyosmani.com/blog/agentic-engineering/

19 20 22 44 Karpathy's March of Nines shows why 90% AI reliability isn't even close to enough | VentureBeat
https://venturebeat.com/technology/karpathys-march-of-nines-shows-why-90-ai-reliability-isnt-even-close-to

21 Graph API overview - Docs by LangChain
https://docs.langchain.com/oss/python/langgraph/graph-api?utm_source=chatgpt.com

23 Working with evals | OpenAI API
https://developers.openai.com/api/docs/guides/evals/?utm_source=chatgpt.com

24 37 Promptfoo: LLM evals & red teaming
https://github.com/promptfoo/promptfoo?utm_source=chatgpt.com

25 Claude Code overview - Claude Code Docs
https://code.claude.com/docs/en/overview?utm_source=chatgpt.com

28 Overview | Cursor Docs
https://cursor.com/docs/agent/overview?utm_source=chatgpt.com

29 GitHub - OpenHands/OpenHands: OpenHands: AI-Driven Development · GitHub
https://github.com/OpenHands/OpenHands

30 GitHub - SWE-agent/mini-swe-agent: The 100 line AI agent that solves GitHub issues or helps you in your command line. Radically simple, no huge configs, no giant monorepo—but scores >74% on SWE-bench verified! · GitHub
https://github.com/SWE-agent/mini-swe-agent

31 Introducing SWE-bench Verified
https://openai.com/index/introducing-swe-bench-verified/?utm_source=chatgpt.com

33 microsoft/autogen: A programming framework for agentic AI
https://github.com/microsoft/autogen?utm_source=chatgpt.com

34 crewAIInc/crewAI: Framework for orchestrating role-playing ...
https://github.com/crewaiinc/crewai?utm_source=chatgpt.com

35 openai/swarm: Educational framework exploring ...
https://github.com/openai/swarm?utm_source=chatgpt.com

36 continuedev/continue: ⏩ Source-controlled AI checks ...
https://github.com/continuedev/continue?utm_source=chatgpt.com

38 GitHub - openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way. · GitHub
https://github.com/openclaw/openclaw

40 Infostealers are being disguised as Claude Code, OpenClaw and other AI developer tools
https://www.techradar.com/pro/security/infostealers-are-being-disguised-as-claude-code-openclaw-and-other-ai-developer-tools?utm_source=chatgpt.com

45 Continue Docs: What is Continue? - Continue.dev
https://docs.continue.dev/?utm_source=chatgpt.com

46 langfuse/langfuse: 🪢 Open source LLM engineering ...
https://github.com/langfuse/langfuse?utm_source=chatgpt.com

49 Quickstart - Claude Code Docs
https://code.claude.com/docs/en/quickstart